

1. Search 基础

world state describes the actual condition of the environment

search state is a representation of the information needed by the search algorithm

fringe the set of generated nodes that have not yet been expanded

2. Uninformed Search

Uninformed search uses no heuristic information about the goal

A **heuristic** is a function that estimates the remaining cost from a node to the goal.

Depth-First Search Go deep first, then backtrack

1.Put the start node in the fringe. 2.Remove the deepest node from the fringe. 3.Check whether it is the goal. 4.If not, expand it and add its children to the fringe. 5.Repeat until the goal is found or the fringe is empty.

-**Completeness** No DFS may follow an infinite path or get stuck in a loop.

-**Optimality** No DFS finds the first goal, which may not be the shallowest or cheapest goal.

-**Time Complexity** $O(b^m)$

-**Space Complexity** $O(bm)$

Breadth-First Search Search level by level

2.Remove the shallowest node from the fringe.

-**Completeness** Yes, if b is finite BFS will eventually reach the shallowest goal.

-**Optimality** Yes. If all step costs are equal. The shallowest goal uses the fewest steps.

If step costs are different: No. The fewest steps may not have the lowest cost.

-**Time Complexity** $O(b^s)$ s is the depth of the shallowest goal

-**Space Complexity** $O(b^s)$

Uniform Cost Search Expand the cheapest path first

1.Put the start node in the priority queue with cost 0. 2.Remove the node with the lowest path cost. 3.Check whether it is the goal. 4.If not, expand it and calculate the cost of each child. 5.Add or update the children in the priority queue. 6.Repeat until the goal is removed from the queue.

-**Completeness** Yes, if every step cost is positive: $cost \geq \epsilon > 0$ With infinitely many zero-cost actions, UCS may not reach the goal.

-**Optimality** Yes. UCS always expands the path with the lowest total cost first.

-**Time Complexity** $O(b^{1+\lceil C^*/\epsilon \rceil})$ C^* : cost of the optimal solution

-**Space Complexity** $O(b^{1+\lceil C^*/\epsilon \rceil})$ ϵ : smallest positive step cost

3. Informed Search

Informed Search uses heuristic information to guide the search toward the goal.

Greedy Search Choose the node that looks closest to the goal.

2.Choose the node with the smallest heuristic value $h(n)$.

-**Completeness** No Greedy search may follow an infinite path or get stuck in a loop.

-**Optimality** No The node that looks closest to the goal may not give the cheapest solution.

-**Time Complexity** $O(b^m)$

-**Space Complexity** $O(b^m)$

Admissibility An admissible heuristic never overestimates the true cost to the goal. 可以低估, 但不能高估。 $h(n) \leq h^*(n)$

A*Search A chooses the smallest past cost plus estimated future cost.*

1.Put the start node in the fringe. 2.Calculate $f(n) = g(n) + h(n)$ for each node. 3.Choose the node with the smallest $f(n)$. 4.Check whether it is the goal. 5.If not, expand it and calculate the costs of its children. 6.Repeat until the goal is removed from the fringe.

-**Completeness** Yes A is complete if the branching factor is finite and every step cost is positive.*

-**Optimality** Yes A is optimal if the heuristic is admissible.*

-**Time Complexity** $O(b^{*s})$

-**Space Complexity** $O(b^{*s})$

4. Constraint Satisfaction Problems 基础

Constraint Satisfaction Problems assigns values to variables while satisfying all constraints.

Variables + Domains + Constraints

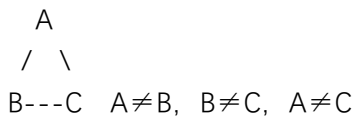
Variables: A, B, C

Domains: {Red, Green, Blue} the set of possible values for a variable

Constraints: $A \neq B$, $B \neq C$, $A \neq C$ is a rule that limits the values of variables

binary constraints is a constraint between two variables

Constraint Graph has variables as nodes and constraints as edges



Assignments mean gives values to variables.

Partial Assignment gives values to some variables $A = \text{Red}$, $B = \text{Green}$

Complete Assignment gives values to all variables $A = \text{Red}$, $B = \text{Green}$, $C = \text{Blue}$

Consistent Assignment does not violate any constraint $A = \text{Red}$, $B = \text{Green}$ satisfy $A \neq B$

5. CSP Search

Backtracking Search assigns values one by one and goes back when a choice fails.

(Try $\rightarrow$ Check $\rightarrow$ Fail $\rightarrow$ Go back)

1. Choose an unassigned variable. **2.** Choose a value from its domain. **3.** Check the constraints. **4.** If the value is valid, continue with the next variable. **5.** If the value causes a conflict, try another value. **6.** If no value works, remove the previous assignment and go back. **7.** Stop when all variables are assigned.

Backtrack means to undo a choice and try another one.

6. CSP Heuristics

Minimum Remaining Values(MRV) chooses the unassigned variable with the fewest legal values(**Most constrained variable**). 选择合法值最少的变量

$D(A) = \{\text{Red}, \text{Green}\}$ $D(B) = \{\text{Blue}\}$ $D(C) = \{\text{Red}, \text{Green}, \text{Blue}\}$ choose B

Least Constraining Value(LCV) chooses the value that removes the fewest choices from other variables.

$D(A) = \{\text{Red}, \text{Green}\}$ $D(B) = \{\text{Red}, \text{Blue}\}$ $D(C) = \{\text{Red}, \text{Green}, \text{Blue}\}$

$A = \text{Red}$ $D(B) = \{\text{Blue}\}$ $D(C) = \{\text{Green}, \text{Blue}\}$ $A = \text{Green}$ $D(B) = \{\text{Red}, \text{Blue}\}$ $D(C) = \{\text{Red}, \text{Blue}\}$ choose $A = \text{Green}$

7. CSP Filtering

Filtering removes values that cannot satisfy the constraints.

$D(A) = \{\text{Red}\}$, $A \neq B$, $A = \text{Red}$ $D(B) = \{\text{Red}, \text{Green}, \text{Blue}\} \rightarrow D(B) = \{\text{Green}, \text{Blue}\}$

Forward Checking removes conflicting values from neighboring variables after an assignment

1. Assign a value to a variable. **2.** Check all unassigned neighboring variables. **3.** Remove values that conflict with the assignment. **4.** If a domain becomes empty, backtrack. **5.** Otherwise, continue with the next variable.

arc consistency An arc is consistent if every value has a supporting value in the neighboring domain.

1. Check an arc $X \rightarrow Y$. **2.** Look at every value in $D(X)$. **3.** Find a supporting value in $D(Y)$. **4.** Remove the value if no support exists. **5.** If a domain changes, check related arcs again. **6.** If a domain becomes empty, backtrack. **7.** Repeat until no more values can be removed.

8. Local Search

Local Search starts with a complete solution and improves it by making small changes.

1. Create a complete initial solution. **2.** Evaluate the current solution. **3.** Make a small change. **4.** Keep the change if it improves the solution. **5.** Repeat until a good solution is found.

Hill Climbing repeatedly moves to a better neighboring state. can get stuck in a local optimum.

Simulated Annealing Accept better moves and sometimes accept worse moves. Lower the temperature over time.

Genetic Algorithms improves a population of solutions through selection, crossover, and mutation.